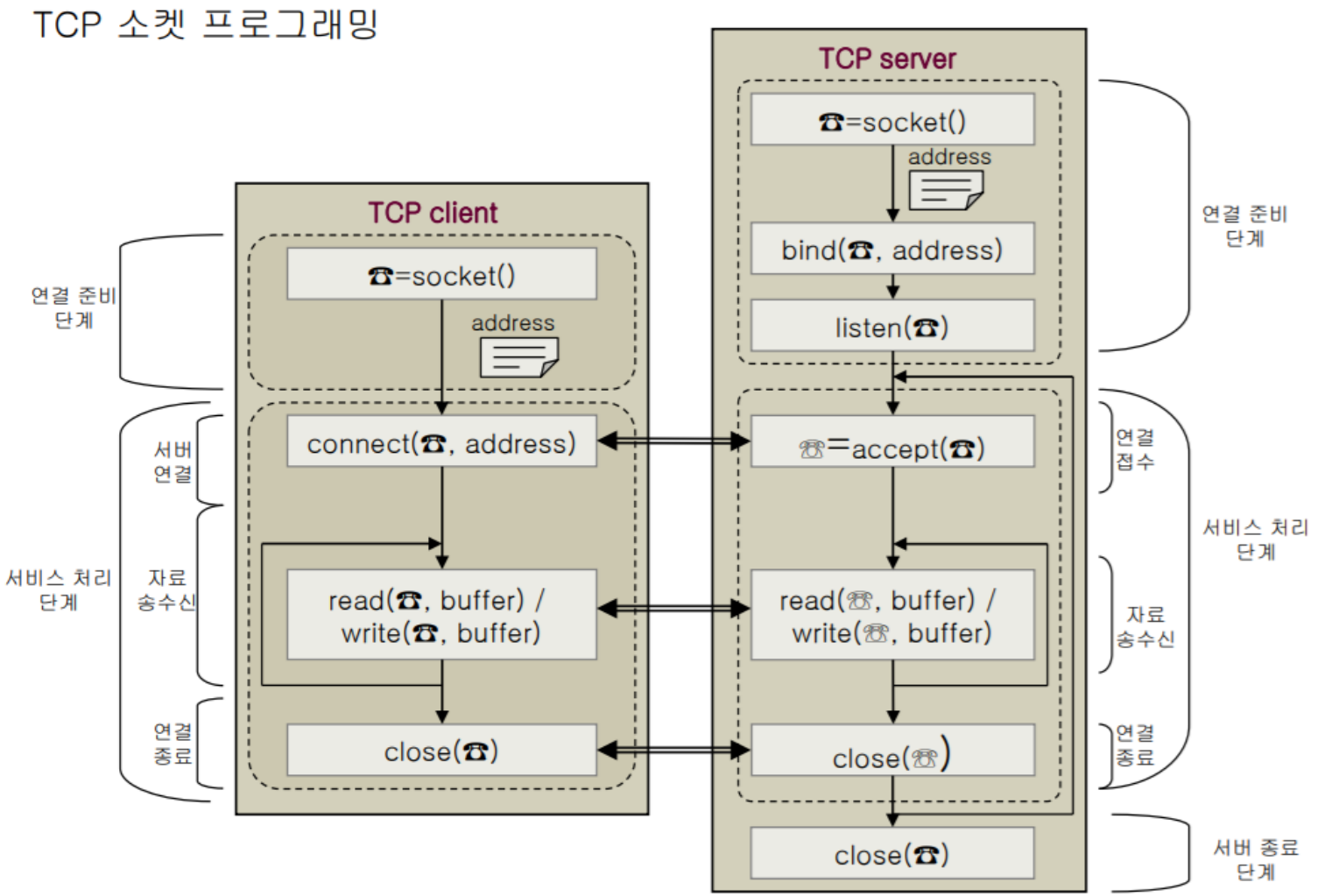
소켓 프로그래밍

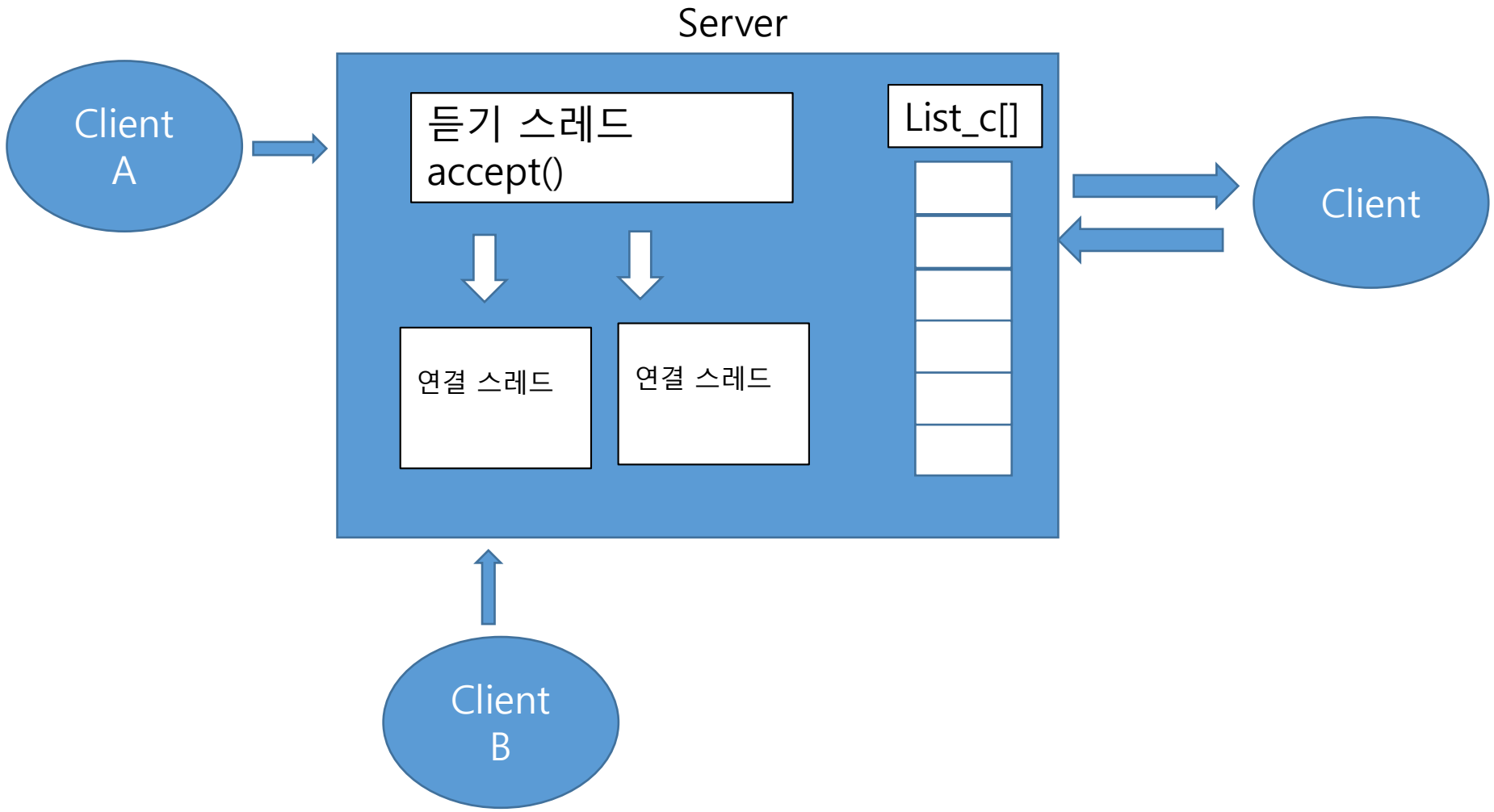
12주 채팅 프로그래밍(14장)

- 멀티쓰레드 (pthread)

소켓 프로그래밍


TCP 소켓 프로그래밍
TCP server
☎=socket()
address
bind(☎, address)
listen(☎)
TCP client
☎=socket()
address
connect(☎, address)
read(☎, buffer) /
write(☎, buffer)
close(☎)
☎=accept(☎)
read(☎, buffer) /
write(☎, buffer)
close(☎)
close(☎)
연결 준비
단계
서버
연결
서비스 처리
단계
자료
송수신
연결
종료
연결 준비
단계
연결
접수
서비스 처리
단계
자료
송수신
연결
종료
서버 종료
단계

소켓 프로그래밍


Server
Client
A
듣기 스레드
accept()
List_c[]
연결 스레드
연결 스레드
Client
Client
B

소켓 프로그래밍

```
// chat_server_thread.c
#include        <stdio.h>
#include        <stdlib.h>
#include        <string.h>
#include        <unistd.h>
#include        <netinet/in.h>
#include        <sys/socket.h>
#include   <pthread.h>

void*                   do_chat(void *);

pthread_t thread;
pthread_mutex_t     mutex;


#define         MAX_CLIENT      10
#define         CHATDATA        1024

#define         INVALID_SOCK        -1

int         list_c[MAX_CLIENT];

char        escape[] = "exit";
char        greeting[] = "Welcome to chatting room₩n";
char        CODE200[] = "Sorry No More Connection₩n";
```

```
main(int argc, char *argv[])
{
        int     c_socket, s_socket;
        struct  sockaddr_in s_addr, c_addr;
        int     len;
        int     nfds = 0;
        int     i, j, n;
        fd_set  read_fds;
             int          res;

        if (argc < 2) {
                printf("usage: %s port_number₩n", argv[0]);
                exit(-1);
        }

             if (pthread_mutex_init(&mutex, NULL) != 0) {
                          printf("Can not create mutex₩n");
                          return -1;
             }

        s_socket = socket(PF_INET, SOCK_STREAM, 0);

        memset(&s_addr, 0, sizeof(s_addr));
        s_addr.sin_addr.s_addr = htonl(INADDR_ANY);
        s_addr.sin_family = AF_INET;
        s_addr.sin_port = htons(atoi(argv[1]));
```

소켓 프로그래밍

```
        if (bind(s_socket, (struct sockaddr *)&s_addr, sizeof(s_addr)) == -1) {
                printf("Can not Bind₩n");
                return -1;
        }

        if(listen(s_socket, MAX_CLIENT) == -1) {
                printf("listen Fail₩n");
                return -1;
        }

                for (i = 0; i < MAX_CLIENT; i++)
                        list_c[i] = INVALID_SOCK;

                while(1) {
                        len = sizeof(c_addr);
                        c_socket = accept(s_socket, (struct sockaddr *)&c_addr, &len);

                        res = pushClient(c_socket);
                        if (res < 0) {
                                write(c_socket, CODE200, strlen(CODE200));
                                close(c_socket);
                        } else {
                                write(c_socket, greeting, strlen(greeting));
                                pthread_create(&thread, NULL, do_chat, (void *) c_socket);
                        }
                }
}
```

```
void *
do_chat(void *arg)
{
        int c_socket = (int) arg;
      char    chatData[CHATDATA];
        int         i, n;

        while(1) {
                memset(chatData, 0, sizeof(chatData));
        if ((n = read(c_socket, chatData, sizeof(chatData))) > 0 ) {
                for (i = 0; i < MAX_CLIENT; i++) {
                if (list_c[i] != INVALID_SOCK) {
                        write(list_c[i], chatData, n);
                                }
                        }

                if(strstr(chatData, escape) != NULL) {
                        popClient(c_socket);
                                break;
                }
        }
      }
}
```

소켓 프로그래밍

```
void *
do_chat(void *arg)
{
        int c_socket = (int) arg;
    char    chatData[CHATDATA];
        int         i, n;

        while(1) {
                memset(chatData, 0, sizeof(chatData));
        if ((n = read(c_socket, chatData, sizeof(chatData))) > 0 ) {
                for (i = 0; i < MAX_CLIENT; i++) {
                if (list_c[i] != INVALID_SOCK) {
                        write(list_c[i], chatData, n);
                                }
                        }

                if(strstr(chatData, escape) != NULL) {
                        popClient(c_socket);
                                break;
                }
        }
    }
}
```

소켓 프로그래밍

```
int
pushClient(int c_socket) {
        int         i;

        for (i = 0; i < MAX_CLIENT; i++) {
                pthread_mutex_lock(&mutex);
                if(list_c[i] == INVALID_SOCK) {
                        list_c[i] = c_socket;
                        pthread_mutex_unlock(&mutex);
                        return i;
                }
                pthread_mutex_unlock(&mutex);
        }

        if (i == MAX_CLIENT)
                return -1;
}
```

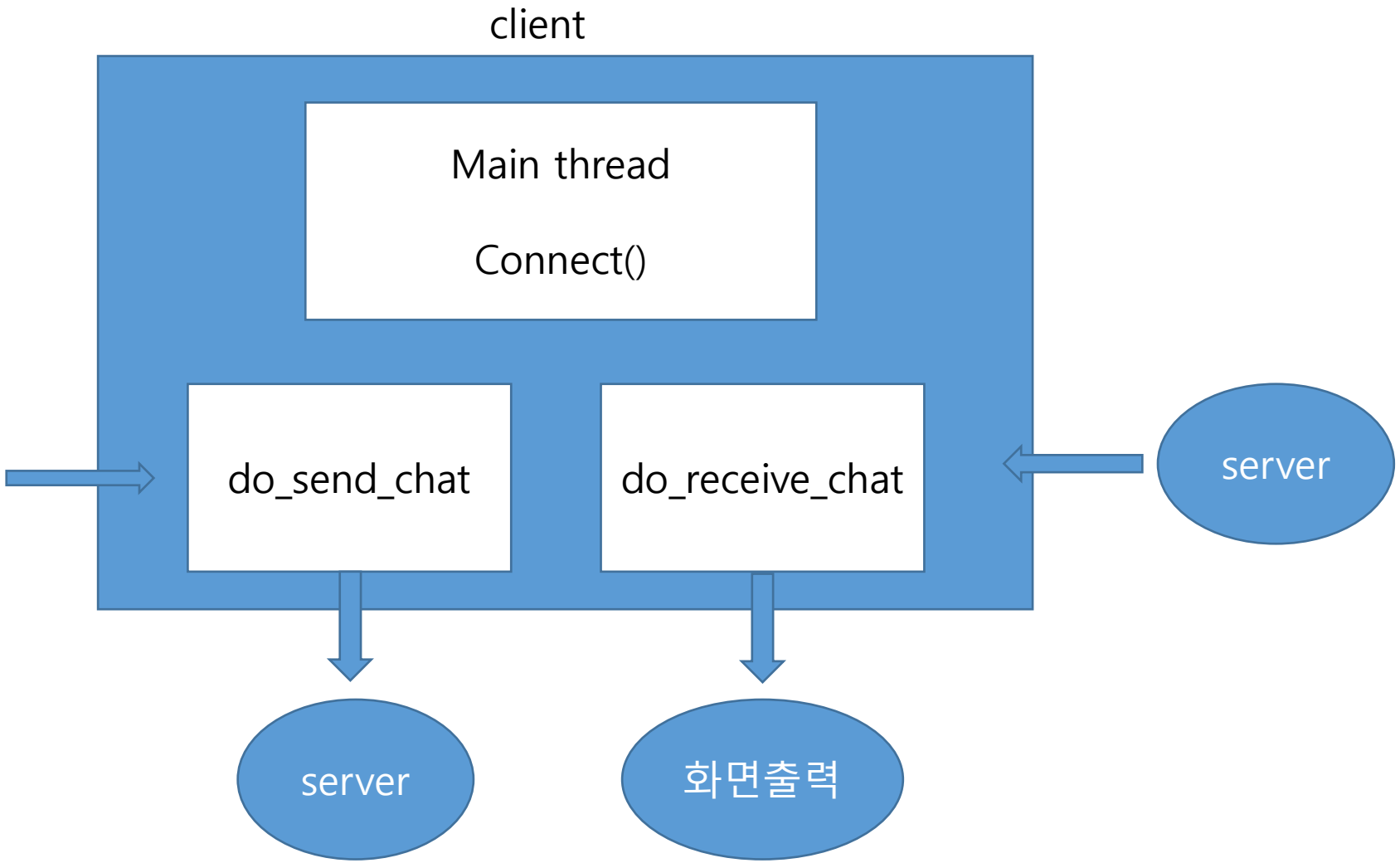
```
int
popClient(int s)
{
        int     i;

        close(s);

        for (i = 0; i < MAX_CLIENT; i++) {
                        pthread_mutex_lock(&mutex);
                if ( s == list_c[i] ) {
                                list_c[i] = INVALID_SOCK;
                                pthread_mutex_unlock(&mutex);
                                break;
                        }
                        pthread_mutex_unlock(&mutex);
        }

        return 0;
}
```

소켓 프로그래밍


client
Main thread
Connect()
do_send_chat
do_receive_chat
server
server
화면출력

소켓 프로그래밍

```
// chat_client_thread.c
#include        <stdio.h>
#include        <stdlib.h>
#include        <string.h>
#include        <unistd.h>
#include        <netinet/in.h>
#include        <sys/socket.h>
#include        <sys/select.h>
#include   <pthread.h>
#include   <signal.h>

#define         CHATDATA        1024

void*                   do_send_chat(void *);
void*                   do_receive_chat(void *);

pthread_t thread_1, thread_2;

char            escape[] = "exit";
char            nickname[20];
```

```
main(int argc, char *argv[])
{
        int      c_socket;
        struct  sockaddr_in c_addr;
        int      len;
        char     chatData[CHATDATA];
        char     buf[CHATDATA];
        int      nfds;
        fd_set  read_fds;
        int      n;

        if (argc < 3) {
                printf("usgae : %s ip_address port_number₩n", argv[0]);
                exit(-1);
        }

        c_socket = socket(PF_INET, SOCK_STREAM, 0);

        memset( &c_addr, 0, sizeof(c_addr) );
        c_addr.sin_addr.s_addr = inet_addr(argv[1]);
        c_addr.sin_family = AF_INET;
        c_addr.sin_port = htons(atoi(argv[2]));

        printf("Input Nickname : ");
        scanf("%s", nickname);
```

소켓 프로그래밍

```
    if (connect(c_socket, (struct sockaddr *)&c_addr, sizeof(c_addr)) == -1) {
        printf("Can not connect₩n");
        return -1;
    }

    pthread_create(&thread_1, NULL, do_send_chat, (void *) c_socket);
    pthread_create(&thread_2, NULL, do_receive_chat, (void *) c_socket);

    pthread_join(thread_1, NULL);
    pthread_join(thread_2, NULL);

    close(c_socket);
}
```

```
void *
do_send_chat(void* arg)
{
        char    chatData[CHATDATA];
        char    buf[CHATDATA];
        int     n;
        int     c_socket = (int) arg;           // client socket

        while(1) {
                memset(buf, 0, sizeof(buf));
          if ((n = read(0, buf, sizeof(buf))) > 0 ) {
                sprintf(chatData, "[%s] %s", nickname, buf);
              write(c_socket, chatData, strlen(chatData));

                if (!strncmp(buf, escape, strlen(escape))) {
                                        pthread_kill(thread_2, SIGINT);
                    break;
                }
          }
        }
}
```

```
void *
do_receive_chat(void* arg)
{
        char    chatData[CHATDATA];
        int     n;
        int     c_socket = (int) arg;           // client socket

        while(1) {
                memset(chatData, 0, sizeof(chatData));
          if ((n = read(c_socket, chatData, sizeof(chatData))) > 0 ) {
                write(1, chatData, n);
          }
        }
}
```

소켓 프로그래밍

```
void *
do_send_chat(void* arg)
{
        char    chatData[CHATDATA];
        char    buf[CHATDATA];
        int     n;
        int     c_socket = (int) arg;          // client socket

        while(1) {
                memset(buf, 0, sizeof(buf));
          if ((n = read(0, buf, sizeof(buf))) > 0 ) {
                sprintf(chatData, "[%s] %s", nickname, buf);
              write(c_socket, chatData, strlen(chatData));

                if (!strncmp(buf, escape, strlen(escape))) {
                                        pthread_kill(thread_2, SIGINT);
                   break;
                }
          }
        }
}
```

소켓 프로그래밍

```
void *
do_receive_chat(void* arg)
{
        char    chatData[CHATDATA];
        int     n;
        int     c_socket = (int) arg;          // client socket

        while(1) {
                memset(chatData, 0, sizeof(chatData));
          if ((n = read(c_socket, chatData, sizeof(chatData))) > 0 ) {
                write(1, chatData, n);
          }
        }
}
```

소켓 프로그래밍

12주 과제 : 채팅서버 기능추가하기(chat_server_thread.c)

1. list_c 배열의 내용을 사용자가 추가된 후와 삭제된 후에 출력한다.
2. 클라이언트가 보낸 내용 중에서 user 이름과 메시지 내용을 분리하여 출력한다.
3. 모든 user에게 보내는 메시지를 특정 유저에게만 전달하도록 한다.

   예) user가 /5/ Hello 라고 보내면 이 메시지는 5번 소켓으로만 보낸다.